

Lab 12 - Resampling

Rita Almeida

Autumn 2025

```
library('tidyverse')
library("boot")
library("MASS")
theme_set(theme_bw())
```

We start by using a data set about adults learning Dutch as a second language in the Netherlands. You can find more about the data here: <https://zenodo.org/records/2863533#.Y9Y3pNJBwUE>. The data set includes test scores from the State Examination of Dutch as a second language, information about their native language as well as demographic information. Below you have a very short description of some of the 16 variables.

L1: First language

C: Country of birth

L2: The second best additional language besides Dutch. If no second language the variable has the value *Monolingual*

L1L2: Variables L1 and L2 together

AaA: Age at arrival in the Netherlands

LoR: Length of residence in the Netherlands

Edu.day: Level of education (1 to 4)

Family: Language family of the first language

Enroll: Proportion of school-aged youth enrolled in secondary education in the country of origin

Speaking: Speaking proficiency test score on the State Examination of Dutch as a Second Language

morph: Morphological dissimilarity from L1 to the target language

lex: Lexical dissimilarity from L1 to the target language

new_feat: Phonological dissimilarity (in terms of new features) from L1 to the target language

new_sounds: Phonological dissimilarity (in terms of new sounds) from L1 to the target language

We load the data from the file *stex.csv*.

```
stex <- read_csv('stex.csv')
```

```
## Rows: 50235 Columns: 16
```

```
## -- Column specification -----
```

```
## Delimiter: ","
```

```
## chr (7): L1, C, L1L2, L2, Sex, Family, IS0639.3
```

```
## dbl (9): AaA, LoR, Edu.day, Enroll, Speaking, morph, lex, new_feat, new_sounds
```

```
##
```

```
## i Use `spec()` to retrieve the full column specification for this data.
```

i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

For simplicity, we select individuals for which there are measurements for all variables. For this we use the command `complete.cases()`.

```
stex <- stex[complete.cases(stex), ]
```

Then we create an extra variable *NotMono* that indicates if the individual is monolingual or not. The variable takes the *True* for those that know more than one language.

```
stex$NotMono <- (stex$L2!="Monolingual")
```

Validation set

For this lab we want to predict the speaking exam score, based on non-exam information about the students and their first language. We will do this using linear regression. To estimate the accuracy of our models we want to split the data set into a *training* data set and a *test* data set, the **validation** data set. Previously, we have done this split by considering the first *n* observations as *training* data and the last as *test* data, or the opposite. But, before doing this, we have a look at the first 10 observations of our data set.

```
head(stex, 10)
```

```
## # A tibble: 10 x 17
##   L1      C    L1L2 L2      AaA  LoR Edu.day Sex  Family IS0639.3 Enroll
##   <chr> <chr> <chr> <chr> <dbl> <dbl> <dbl> <chr> <chr> <chr> <dbl>
## 1 Albanian Albania Alba~ Engl~ 17    3    3 Male Indo~~ sqi    78
## 2 Albanian Albania Alba~ Germ~ 23    1    4 Fema~ Indo~~ sqi    78
## 3 Albanian Yugosl~ Alba~ Croa~ 22    3    3 Fema~ Indo~~ sqi    85
## 4 Albanian Albania Alba~ Engl~ 30    3    4 Fema~ Indo~~ sqi    78
## 5 Albanian Albania Alba~ Greek 17    1    1 Fema~ Indo~~ sqi    78
## 6 Albanian Yugosl~ Alba~ Fren~ 26    4    2 Fema~ Indo~~ sqi    85
## 7 Albanian Yugosl~ Alba~ Engl~ 20    6    3 Male  Indo~~ sqi    85
## 8 Albanian Yugosl~ Alba~ Croa~ 22    4    3 Male  Indo~~ sqi    85
## 9 Albanian Yugosl~ Alba~ Engl~ 22    2    3 Male  Indo~~ sqi    85
## 10 Albanian Yugosl~ Alba~ Engl~ 23    4    4 Fema~ Indo~~ sqi    85
## # i 6 more variables: Speaking <dbl>, morph <dbl>, lex <dbl>, new_feat <dbl>,
## #   new_sounds <dbl>, NotMono <lgl>
```

It turns out that the data is ordered according to the variable *L1*, and all the first 10 individuals have Albanian as first language. Due to this ordering, if we use the method of splitting the data in the middle, both the *training* data and the *test* data will not be representative of the full sample. When the training sample does not represent well the actual population for which we want to make predictions, we will not get a good model. Instead, what we need to do is to select a training data set by sampling at random from the total data. We do this by using the command *sample* to which we give as inputs: the total number of observations and the size of the sample we want to draw.

```
set.seed(11)
index.train1 = sample (47232, round(47232/2))
```

Then we can create the training (*stex.train1*) and test (*stex.test1*) data sets.

```
stex.train1 <-stex[index.train1, ]
stex.test1 <-stex[-index.train1, ]
```

Next we fit a linear model using the command *lm* on the training data and use the command *summary* to get some information about the model fit. We start by fitting a model that only contains information about the individuals and education in their country of origin.

```
stex.m1 <- lm(Speaking~ AaA + LoR +Edu.day +  Enroll + factor(NotMono) ,
              data=stex.train1)
summary(stex.m1)
```

```
##
## Call:
## lm(formula = Speaking ~ AaA + LoR + Edu.day + Enroll + factor(NotMono),
##     data = stex.train1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -250.81  -21.24   -1.01   20.35  169.03
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    476.279767    1.446215  329.328 < 2e-16 ***
## AaA             -0.751140    0.031986  -23.484 < 2e-16 ***
## LoR              0.249784    0.054632   4.572 4.85e-06 ***
## Edu.day         1.695609    0.269898   6.282 3.39e-10 ***
## Enroll          0.564203    0.008887  63.487 < 2e-16 ***
## factor(NotMono)TRUE 11.003958    0.595420  18.481 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 34.39 on 23610 degrees of freedom
## Multiple R-squared:  0.1725, Adjusted R-squared:  0.1723
## F-statistic: 984.5 on 5 and 23610 DF,  p-value: < 2.2e-16
```

The *predict* function is used to estimate the speaking exam score for the test data.

```
stex.test1$pred1 <- (predict(stex.m1, newdata=stex.test1))
```

Now we have everything to calculate the training and test mean square error (MSE).

```
MSE.train1 = mean(stex.m1$residuals^2)
MSE.test1 = mean((stex.test1$Speaking - stex.test1$pred1)^2)
MSE.train1
```

```
## [1] 1182.41
```

```
MSE.test1
```

```
## [1] 1193.588
```

Likely, we get a test MSE larger than the training MSE. If we draw another sample we will get different estimates of MSE both for the training and the test data sets. To check this, we repeat the procedure.

We draw a new sample.

```
index.train2 = sample (47232, round(47232/2))
```

We create a new train set and a new test set.

```
stex.train2 <-stex[index.train2, ]
stex.test2 <-stex[-index.train2, ]
```

We fit the model

```
stex.m2 <- lm(Speaking~ AaA + LoR + Edu.day + Enroll + factor(NotMono) ,
              data=stex.train2)
```

We apply it to the test data.

```
stex.test2$pred2 <- (predict(stex.m2, newdata=stex.test2))
```

Now we have everything to calculate the train and test mean square error (MSE).

```
MSE.train2 = mean(stex.m2$residuals^2)
MSE.test2 = mean((stex.test2$Speaking - stex.test2$pred2)^2)
MSE.train2
```

```
## [1] 1187.606
```

```
MSE.test2
```

```
## [1] 1188.384
```

You can see that you got different results, but most likely not very different, since this is quite a large data set.

Exercise

We start by creating a subset *stexIE* of the original data, containing only individuals that have an indo-european first language, that is those for which the variable *Family* takes the value *Indo-European*.

```
stexIE <- stex[stex$Family=="Indo-European",]
```

This is the largest language family group in the data set, and *stex.IE* contains 32955 observations.

There are 28 different first languages in this subset. We can see how the distribution of individuals for the different first languages *L1* is.

```
table(stexIE[, 'L1'])
```

```
## L1
##      Albanian      Armenian      Bosnian      Bulgarian Byelorussian      Catalan
##          288          625           99          562           26           70
##      Croatian      Czech      English      French      German      Greek
##          481          471         3040         1574         5226         333
##          Hindi      Icelandic      Italian      Kurdish      Latvian      Lithuanian
##          100           41          661          1126           75          192
##      Norwegian      Persian      Polish      Portugese      Romanian      Russian
##          203          3382         2651         1255           990         3770
##          Serbian      Spanish      Swedish      Ukranian
##          2188          2872          312          342
```

```
dim(unique(stexIE[, 'L1']))
```

```
## [1] 28  1
```

Now use the validation set approach to estimate the test error rates on the data set *stexIE* for a model with the same explanatory variables that you used in the model of the previous section. Then repeat the procedure but for a model including the variable *L1* too. Remember that the variable *L1* is categorical. Comment the results.

```
#write your solution here
```

Leave-one-out cross-validation

We have used the function *lm()* to fit linear regressions. These models can also be fit using the function *glm()*, which we used in the previous lab to fit logistic models. To fit a linear regression with *glm()* one does not pass

a value to the option *family*. For logistic regression we had used *family*="binomial". Here we will use *glm()* because the leave-one-out cross-validation estimator can be directly calculated using the function *cv.glm()*.

In this section we will use a subset of the *stexIE* data we created above. We select a set with 2000 randomly sampled individuals, to speed up the calculations below.

```
set.seed(2)
ii = sample (32955,2000)
stexIEx <- stexIE[ii, ]
```

Then we fit a model with the same variables we used for the first model we fitted.

```
stexIEx.glm1 <- glm(Speaking~ AaA + LoR + Enroll + Edu.day+ factor(NotMono),
                   data=stexIEx)
```

Now the command *cv.glm* is used to calculate the leave-one-out cross-validation estimator. This would take some minutes. Wait until the calculations are finished before you proceed.

```
cv.error1 = cv.glm(stexIEx,stexIEx.glm1)
```

The estimator is saved on the first column of the output *delta*.

```
cv.error1$delta[1]
```

```
## [1] 1159.339
```

We want to extend the model to include measures about the dissimilarity between the individuals first language and the target language, that is Dutch. We start by fitting a model that apart from the previous variables also includes the variable *morph*.

```
stexIEx.glm2 <- glm(Speaking~ AaA + LoR + Enroll + Edu.day+ factor(NotMono) +
                   morph , data=stexIEx)
```

Then we calculate the leave-one-out cross-validation estimator for the error rate of this model.

```
cv.error2 = cv.glm(stexIEx,stexIEx.glm2)
cv.error2$delta[1]
```

```
## [1] 1064.627
```

One can observe that the test error rate gets smaller. We proceed by fitting models to which we successively add the other language dissimilarity measures: *lex*, *new_feat* and *new_sounds*.

```
stexIEx.glm3 <- glm(Speaking~ AaA + LoR + Enroll + Edu.day+ factor(NotMono) +
                   morph + lex , data=stexIEx)
cv.error3 = cv.glm(stexIEx,stexIEx.glm3)
cv.error3$delta[1]
```

```
## [1] 1021.865
```

```
stexIEx.glm4 <- glm(Speaking~ AaA + LoR + Enroll + Edu.day+ factor(NotMono) +
                   morph + lex +new_feat , data=stexIEx)
cv.error4 = cv.glm(stexIEx,stexIEx.glm4)
cv.error4$delta[1]
```

```
## [1] 1023.007
```

```
stexIEx.glm5 <- glm(Speaking~ AaA + LoR + Enroll + Edu.day+ factor(NotMono) +
                   morph + lex +new_feat + new_sounds, data=stexIEx)
cv.error5 = cv.glm(stexIEx,stexIEx.glm5)
cv.error5$delta[1]
```

```
## [1] 1021.198
```

We see a large decrease in error rate when we introduce the variables *morph* and *lex*. Further adding *new_feat* and *new_sounds* does not lead to large decreases in the estimated error rate.

k-fold cross-validation

k-fold cross-validation estimators of the error rate can also be implemented with the function `cv.glm()`. We use this method with the data subset *stexIEx*, and the same models as above. Here we use $k = 10$, which is a quite common value to choose.

For the model *stexIE.glm1* we get:

```
cv.error.k10.1 = cv.glm(stexIEx,stexIEx.glm1,K=10)
cv.error.k10.1$delta[1]
```

```
## [1] 1159.388
```

And for the other models:

```
cv.error.k10.2 = cv.glm(stexIEx,stexIEx.glm2,K=10)
cv.error.k10.2$delta[1]
```

```
## [1] 1064.634
```

```
cv.error.k10.3 = cv.glm(stexIEx,stexIEx.glm3,K=10)
cv.error.k10.3$delta[1]
```

```
## [1] 1021.532
```

```
cv.error.k10.4 = cv.glm(stexIEx,stexIEx.glm4,K=10)
cv.error.k10.4$delta[1]
```

```
## [1] 1026.892
```

```
cv.error.k10.5 = cv.glm(stexIEx,stexIEx.glm5,K=10)
cv.error.k10.5$delta[1]
```

```
## [1] 1020.135
```

One can see that the estimates are very close to those computed with leave-one-out cross-validation. However, the k-fold cross-validation estimators were much faster to obtain.

Exercise: cross-validation with logistic regression

As mentioned above, the function `cv.glm()` works with models fitted with `glm()`, such as the logistic regressions we fitted on the previous lab. Use the data from that lab, in the file *hassemer_winter_2016_gesture.csv*, and fit the same two models as before. The first model has only one predictor *pinkie_curl*. The second model has the three predictors: *pinkie_curl*, *pinkie_sq* and *index_curve*. Estimate the error rates for both models using 10-fold cross-validation. Comment the results.

```
# write your solution here
```

Bootstrap

In this section we will see how the bootstrap approach can be used to estimate the variability of the coefficient estimates from a linear regression. We use the data set *stextIEx* with 2000 observations. We fit a very simple model, just an intercept and the variable with the number of years of residency in the Netherlands *LoR*.

To start, we create a function in R, that takes the data set and returns the coefficient estimates for the intercept and the variable *LoR*.

```
boot.fn=function(data,index){
  +return(coef(lm(Speaking~LoR,data=data,subset=index)))
}
```

And now we apply the *boot.fn* function to the dataset to get the estimates based on the actual data. Here the model is fitted to all the data, since the index vector contains all the values 1 to 2000.

```
boot.fn(stexIEx,1:2000)
```

```
## (Intercept)          LoR
## 525.6636985    0.1301333
```

The function can be applied to new data sets obtained by sampling with replacement from the original data set, the bootstraps. The samples can be obtained using the function *sample* with the argument *replace=T* indicating that the samples are taken with replacement. Below you can see two examples:

```
boot.fn(stexIEx,sample(2000,2000,replace=T))
```

```
## (Intercept)          LoR
## 525.1869225    0.3058921
```

```
boot.fn(stexIEx,sample(2000,2000,replace=T))
```

```
## (Intercept)          LoR
## 524.0947140    0.5564748
```

To estimate the standard error of the coefficients using bootstrap we could repeat this process 1000 times and then calculate the standard deviation over all the coefficient estimates. However, we can use the *boot()* function as shown below.

```
boot(stexIEx,boot.fn,1000)
```

```
##
## ORDINARY NONPARAMETRIC BOOTSTRAP
##
## Call:
## boot(data = stexIEx, statistic = boot.fn, R = 1000)
##
##
## Bootstrap Statistics :
##      original      bias    std. error
## t1* 525.6636985 -0.03738035   1.1597582
## t2*   0.1301333  0.01235506   0.2178499
```

The output gives us both the estimates of the coefficients and their standard error.

To see how the bootstrap estimates compare with the ones calculated with the standard regression formulas we can use *summary()*\$*coef* to see the results of the linear regression model on the original data.

```
model.test <- lm(Speaking~LoR,data=stexIEx)
summary(model.test)$coef
```

```
##              Estimate Std. Error    t value Pr(>|t|)
## (Intercept) 525.6636985  1.1010918 477.4022254 0.0000000
## LoR         0.1301333   0.2001754   0.6500966 0.5157046
```

One can see that the bootstrap estimates are close to the ones from the standard formulas. The bootstrap estimates have the general advantage of not relying on any assumptions about the data, but only on the data itself. Here the standard error of the parameter estimates are calculated just based on the values of the

parameters estimated outputed for each re-sample. They are not based on theoretical knowledge about the parameters distribution. Hence, this sort of estimates can be applied even for statistics for which we do not have analytical expressions.